

AI 사용 보고서

YOLO26n 기반 실시간 객체 탐지 과제

학번 : 2026406076

이름 : 정창규

제출일 : 2026-09-28

목차

- 1. AI 사용 개요**
- 2. AI를 사용한 부분과 목적**
 - 2.1 YOLO와 COCO 데이터셋 이해
 - 2.2 ROS2 노드 구성
 - 2.3 YAML 파일을 이용한 설정
 - 2.4 영상 끊김 문제 해결
 - 2.5 실행 환경과 오류 해결
- 3. 작업 과정에서 배운 점**
- 4. AI 사용 후 느낀 점**
- 5. 최종 구현 결과**

1. AI 사용 개요

이번 과제에서는 COCO 데이터셋으로 사전 학습된 YOLO26n 모델을 이용하여 노트북 웹캠 영상에서 객체를 탐지하는 프로그램을 구현하였다. 카메라 영상을 받아오는 노드와 YOLO 추론을 수행하는 노드를 분리하고, 두 노드가 ROS2 토픽을 통해 영상을 주고받도록 구성하였다.

과제를 진행하면서 AI는 YOLO와 COCO 데이터셋의 관계를 이해하거나 코드 실행 중 발생한 오류의 원인을 찾는 데 사용하였다. 특히 처음에는 웹캠에 물체를 보여주고 YOLO가 이를 인식하는 과정이 학습인지 추론인지 정확히 구분하지 못했는데, AI에게 질문하면서 이미 학습된 yolo26n.pt 모델을 불러와 새로운 영상에 대해 결과를 계산하는 과정이 추론이라는 점을 이해할 수 있었다. 자료에서도 YOLO 추론은 학습된 가중치를 사용하며, COCO는 객체 탐지용 80개 클래스를 포함한다고 설명하고 있다.

또한 처음부터 완성된 코드를 한 번에 사용하는 방식보다는 실제로 실행한 뒤 발생한 문제를 하나씩 수정하는 과정에서 AI를 활용하였다.

2. AI를 사용한 부분과 목적

2.1 YOLO와 COCO 데이터셋 이해

YOLO26n 모델이 어떤 기준으로 사람이나 물체를 판단하는지 이해하기 위해 AI에게 질문하였다. 처음에는 웹캠 영상을 입력하는 과정에서 모델이 새롭게 학습되는 것인지, 기존에 학습된 결과를 이용하는 것인지 정확히 구분하지 못했다. AI에게 질문하면서 yolo26n.pt 파일에 학습된 가중치가 저장되어 있으며, 웹캠 영상에서는 이 가중치를 이용하여 객체를 판단하는 추론이 수행된다는 점을 이해하였다.

이번 과제에서는 별도의 데이터 수집이나 라벨링, 모델 학습을 진행하지 않았다. 이미 COCO로 학습된 모델을 이용하여 person, bottle, cup, keyboard, cell phone, book 등의 클래스만 선택해 탐지하도록 구성하였다. COCO에 포함되지 않은 대상을 탐지하려면 별도의 학습이 필요하다는 내용도 교육 자료를 통해 확인하였다.

YOLO26n으로 웹캠 영상을 인식하는 건 학습이야 추론이야? 차이도 같이 설명해줘.

YOLO26n으로 웹캠 영상을 인식하는 과정은 ****학습이 아니라 추론(Inference)****이야.

YOLO26n은 이미 COCO 데이터셋을 이용해 학습된 모델이고, `yolo26n.pt` 파일에는 학습된 결과인 가중치가 저장되어 있어. 웹캠 영상을 입력하면 이 가중치를 이용해서 현재 화면에 어떤 객체가 있는지 판단해.

- **학습(Training)**: 이미지와 정답 데이터를 이용해 모델의 가중치를 수정하는 과정
- **추론(Inference)**: 이미 학습된 가중치를 이용해 새로운 이미지에서 객체를 찾아내는 과정

따라서 웹캠에 사람이나 물체를 보여주고 `person`, `bottle`, `cup` 등으로 인식하는 것은 새로 학습시키는 것이 아니라 **이미 학습된 YOLO26n 모델을 이용해 실시간으로 추론하는 과정**이야.

yolo26n.pt 파일에는 무엇이 들어 있고 웹캠 객체 탐지에서 이 파일을 어떻게 사용하는 거야?

yolo26n.pt 파일에는 YOLO26n이 학습하면서 얻은 **가중치(weight)**와 모델 실행에 필요한 정보가 저장되어 있어.

쉽게 말하면 COCO 데이터셋으로 학습한 결과가 숫자 형태로 정리되어 들어 있는 파일이야. 웹캠 객체 탐지에서는 이 파일을 불러온 뒤, 카메라에서 들어오는 각 프레임들을 모델에 입력해서 어떤 객체가 있는지 판단해.

흐름은 이렇게 보면 돼.

```

</> 일반 텍스트
웹캠 프레임
↓
yolo26n.pt 모델에 입력
↓
학습된 가중치를 이용해 특징 분석
↓
객체 클래스, 위치, confidence 출력

```

즉 yolo26n.pt 는 웹캠 영상을 새로 학습하는 데 사용하는 파싱이 아니라, 이미 학습된 결과를 이용해서 추론하기 위한 모델 파일이야.

2.2 ROS2 노드 구성

카메라 입력과 객체 탐지를 하나의 프로그램에서 처리하지 않고 camera_node와 yolo_node로 나누었다. 과제에서도 카메라 노드와 추론 노드를 분리하고 YAML 설정 및 Launch 실행을 사용하도록 안내되어 있다.

camera_node에서는 OpenCV를 이용하여 노트북 웹캠 영상을 읽고, CvBridge를 사용하여 OpenCV 이미지를 ROS2의 sensor_msgs/Image 형식으로 변환하였다. 변환된 영상은 /camera/image_raw 토픽으로 전송하였다.

yolo_node에서는 해당 토픽을 구독한 뒤 YOLO26n 모델에 영상을 입력하여 객체를 탐지하였다. 탐지된 객체에는 Bounding Box를 그리고, 박스 위에는 클래스 이름과 confidence 값을 함께 표시하였다. 추론에 걸린 시간은 영상 위가 아니라 터미널에 출력하도록 구현하였다.

이 과정을 통해 publisher와 subscriber가 단순한 개념이 아니라 실제 영상 데이터를 서로 전달하는 데 사용된다는 점을 이해할 수 있었다.

ROS2에서 카메라 노드와 YOLO 추론 노드를 왜 따로 분리해서 사용하는 거야? 토픽을 통해 영상을 전달하는 방식도 같이 설명해줘.

카메라 노드와 YOLO 추론 노드를 분리하는 이유는 각 노드의 역할을 명확하게 나누기 위해서야.

카메라 노드는 웹캠 영상을 읽어서 ROS2의 이미지 메시지로 변환한 뒤, 지정한 토픽으로 계속 전송해. YOLO 추론 노드는 그 토픽을 구독하고 있다가 영상이 들어오면 해당 프레임을 YOLO26n 모델에 넣어서 객체를 탐지해.

동작 흐름은 이렇게 보면 돼.

```

</> 일반 텍스트
웹캠
↓
camera_node
↓
/camera/image_raw 토픽
↓
yolo_node
↓
YOLO26n 객체 탐지

```

이렇게 분리하면 카메라와 YOLO의 실행 주기를 각각 다르게 설정할 수 있고, 한쪽 코드를 수정해도 다른 쪽에 영향을 덜 주기 때문에 관리하기도 편해.

즉, 카메라 노드는 영상 전달, YOLO 노드는 객체 탐지 역할을 담당하고 두 노드는 ROS2 토픽을 통해 서로 연결되는 구조야.

2.3 YAML 파일을 이용한 설정

YAML 파일에는 카메라 토픽 이름, 카메라 실행 주기, 모델 이름, confidence 값, YOLO 추론 주기, 탐지할 클래스 ID와 클래스 이름 등을 저장하였다. 이를 통해 Python 코드를 직접 수정하지 않고도 설정값만 변경하여 프로그램의 동작을 바꿀 수 있도록 하였다.

예를 들어 탐지할 객체를 변경할 때는 class_ids와 class_names만 수정하면 되도록 구성하였다. COCO 모델 출력은 클래스 인덱스를 기준으로 구분되기 때문에 ID와 클래스 이름을 함께 관리하였다.

2.4 영상 끊김 문제 해결

객체 탐지 자체는 정상적으로 동작하였지만 처음 실행했을 때 웹캠 화면이 끊겨 보이는 문제가 있었다. 터미널의 추론시간을 확인해 보니 YOLO 추론에 수십 ms 이상의 시간이 필요했고, 추론을 수행하는 동안 영상 표시에도 영향을 주고 있었다.

AI에게 이 문제를 질문하면서 카메라의 Hz와 YOLO 추론 Hz가 서로 다른 역할을 한다는 점을 이해하였다. 카메라는 높은 주기로 영상을 계속 받아오고, YOLO는 CPU 성능에 맞춰 더 낮은 주기로 추론하도록 변경하였다. 또한 추론을 별도의 스레드에서 수행하여 영상 표시가 YOLO 추론이 끝날 때까지 기다리지 않도록 수정하였다.

ROS2 QoS 설정도 오래된 프레임이 계속 쌓이지 않도록 최신 프레임 위주로 처리하도록 조정하였다. 이후 처음보다 화면이 부드럽게 출력되는 것을 확인할 수 있었다.

이 과정에서 단순히 Hz를 높인다고 프로그램이 더 빠르게 동작하는 것은 아니며, CPU가 실제로 처리할 수 있는 속도를 고려해야 한다는 점을 알게 되었다.

YOLO 돌리니까 객체 탐지는 되는데 웹캠 화면이 계속 끊겨. 카메라는 30Hz로 하고 YOLO는 8Hz로 돌리고 있는데 왜 그런 거야?

📄 ✎

그 설정이어도 화면이 끊길 수 있어. 카메라가 30Hz라고 해서 실제 화면이 항상 30fps로 처리되는 건 아니기 때문이야.

카메라 30Hz는 약 33ms마다 한 프레임을 받아오겠다는 의미인데, YOLO 추론 한 번에 수십 ms가 걸리고 그동안 CPU 사용량이 크게 올라가면 카메라 프레임 처리나 화면 출력도 같이 늦어질 수 있어. YOLO를 8Hz로 실행하더라도 한 번 추론할 때 CPU를 많이 사용하는 건 그대로야.

또 ROS2에서 이전 프레임이 큐에 계속 쌓이면 YOLO가 최신 화면이 아니라 늦게 들어온 프레임을 처리하면서 화면이 더 끊겨 보일 수도 있어.

그래서 이런 경우에는 카메라 화면 처리는 계속 빠르게 유지하고, YOLO 추론은 별도로 실행해서 둘이 서로 기다리지 않게 하는 방식이 좋아. 그리고 이미지 큐는 최신 프레임만 남기도록 줄이고, 필요하면 YOLO 입력 이미지 크기나 추론 주기도 낮추면 CPU 부담을 줄일 수 있어.

2.5 실행 환경과 오류 해결

YOLO를 처음 실행했을 때 `ModuleNotFoundError: No module named 'ultralytics'` 오류가 발생하였다. `yolo26n.pt` 파일은 이미 다운로드한 상태였지만, `.pt` 파일과 이를 실행하기 위한 `Ultralytics` 라이브러리는 서로 다른 것이라는 점을 이 과정에서 알게 되었다.

Ubuntu에서 Python 가상환경을 만든 뒤 `Ultralytics`를 설치하였고, ROS2의 `rclpy`와 `cv_bridge`도 함께 사용할 수 있도록 설정하였다. 이후에도 시스템 Python과 가상환경 Python이 서로 달라 YOLO 노드가 실행되지 않는 문제가 있었는데, 가상환경을 활성화한 상태에서 다시 빌드하면서 해결하였다.

이 오류를 해결하면서 프로그램 코드뿐만 아니라 Python 환경과 설치된 라이브러리도 실행 결과에 큰 영향을 준다는 점을 알 수 있었다.

`yolo26n.pt` 파일은 있는데 실행하니까 `ModuleNotFoundError: No module named 'ultralytics'` 오류가 떠 왜 이런 거야?

`yolo26n.pt` 파일이 있어도 `ultralytics` 라이브러리가 설치되어 있지 않으면 그 오류가 발생할 수 있어.

`yolo26n.pt` 는 학습된 모델의 가중치가 저장된 파일이고, 이 파일을 실제로 불러와서 추론을 실행하는 역할은 `ultralytics` 라이브러리가 담당해. 그래서 모델 파일만 있다고 바로 실행되는 건 아니야.

즉 역할을 나누면,

- `yolo26n.pt` : 학습된 모델 정보와 가중치
- `ultralytics` : 모델을 불러오고 입력 영상을 넣어 추론을 실행하는 Python 라이브러리

이런 관계라고 보면 돼.

또 `ultralytics` 를 설치했는데도 같은 오류가 난다면, 설치한 Python 환경과 ROS2 노드가 실제로 실행되는 Python 환경이 서로 다른 경우도 확인해야 해. 예를 들어 가상환경에 설치했는데 ROS2가 시스템 Python으로 실행되면 `ultralytics` 를 찾지 못 할 수 있어.

3. 작업 과정에서 배운 점

이번 과제를 통해 가장 먼저 학습과 추론의 차이를 이해하게 되었다. 웹캠에 새로운 물체를 보여주는 것이 새로운 학습 과정이 아니라, 이미 학습된 YOLO26n 모델을 이용하여 현재 영상의 객체를 판단하는 추론이라는 점을 알게 되었다.

ROS2의 노드와 토픽 구조도 이전보다 구체적으로 이해할 수 있었다. 카메라 노드에서 영상을 보내고 YOLO 노드에서 이를 받아 처리하는 과정을 직접 구현하면서 `publisher`와 `subscriber`가 실제 프로그램에서 어떻게 연결되는지 확인할 수 있었다.

YAML 파일을 이용하여 설정값을 코드와 분리하는 방법도 배웠다. 토픽 이름이나 실행 주기, 탐지할 클래스를 변경할 때마다 Python 코드를 수정할 필요가 없다는 점이 편리하였다.

가장 많이 고민한 부분은 영상 끊김 문제였다. 처음에는 카메라 Hz를 높이면 무조건 화면이 부드러워질 것이라고 생각했지만, YOLO 추론이 CPU를 많이 사용하는 상황에서는 추론 주기를 너무 높이면 오히려 전체 프로그램이 느려질 수 있다는 점을 확인하였다. 카메라 영상 처리와 YOLO 추론 속도를 따로 생각해야 한다는 점이 이번 과제에서 새롭게 이해한 부분이었다.

또한 터미널에 추론 지연시간을 출력하면서 객체가 정상적으로 탐지되는지만 확인하는 것뿐 아니라 실제 처리 속도도 중요하다는 점을 알게 되었다. 교육 자료에서도 추론 결과를 검증할 때 클래스, 점수, 좌표, 탐지 수 등을 확인하도록 설명하고 있다.

4. AI 사용 후 느낀 점

이번 과제에서는 AI를 주로 이해되지 않는 개념을 질문하고, 실행 중 발생한 오류를 해결하는 데 사용하였다. 특히 오류가 발생했을 때 단순히 수정된 코드만 확인하는 것보다 오류 메시지가 왜 발생했는지를 같이 확인하는 과정이 도움이 되었다.

또한 같은 기능을 구현하더라도 코드 구조나 설정 방법에 따라 실행 결과가 달라질 수 있다는 점을 알게 되었다. 처음 작성한 코드를 그대로 사용하는 것이 아니라 직접 실행하고, 화면 끊김이나 라이브러리 오류처럼 실제로 발생한 문제를 하나씩 수정하면서 코드의 역할을 더 이해할 수 있었다.

앞으로도 AI를 사용할 때 완성된 결과만 받아 사용하는 방식보다는 수업 자료를 먼저 확인하고, 이해하지 못한 부분이나 오류가 발생한 이유를 질문하는 방식으로 활용하는 것이 더 도움이 될 것이라고 생각하였다.

5. 최종 구현 결과

최종적으로 노트북 웹캠 영상을 camera_node에서 입력받아 ROS2 토픽으로 전송하고, yolo_node에서 YOLO26n 모델을 이용하여 실시간 객체 탐지를 수행하였다.

탐지 결과에는 Bounding Box와 클래스 이름, confidence 값을 표시하였고, 추론 지연시간은 터미널에서 확인할 수 있도록 하였다. 또한 YAML 파일에서 토픽 이름, 실행 주기, 모델, confidence, 탐지할 클래스를 설정할 수 있도록 구성하였으며, Launch 파일 하나로 카메라 노드와 YOLO 노드를 동시에 실행할 수 있도록 하였다. 이는 과제에서 요구한 구현 항목과 일치한다.